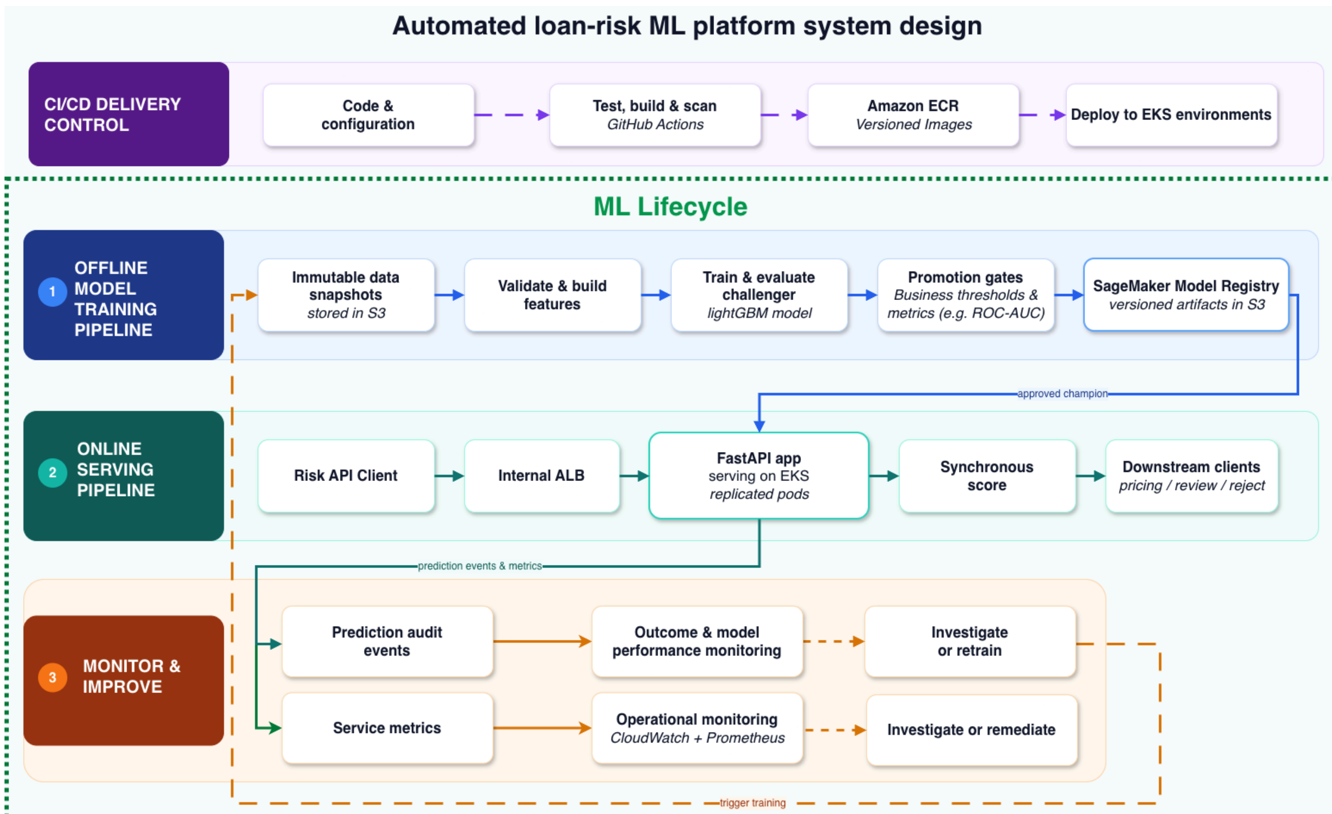
Automated Loan-Risk ML System Design

A production-oriented ML lifecycle for continuous LightGBM updates can be grounded by a concise operational context: the model is scored before pricing, uses application-time features only, and should support immutable challenger training, gated promotion, and safe online inference.

Three ML pipelines and the CI/CD control plane



The dashed boundary is the ML lifecycle. It contains three connected pipelines, each with a distinct operational responsibility:

1. **Offline model-training pipeline:** Immutable S3 snapshots are validated and transformed, then a challenger is trained and evaluated. Promotion gates check the candidate before its versioned artifacts are registered as the approved champion.

2. **Online serving pipeline**: The risk API client reaches the FastAPI service through the internal ALB. The service loads only the approved champion, returns a synchronous score for the downstream pricing, review, or reject workflow, and exposes readiness and service metrics.

3. **Monitor and improve pipeline:** Prediction audit events, service metrics, and mature outcomes feed performance and operational monitoring. An alert leads to investigation, remediation, or a controlled retraining trigger; it does not change the live model automatically.

Failure	System response
Duplicate keys, missing fields, or an invalid join	Stop the training run and quarantine the invalid data.
Candidate artifact or serving contract is incompatible	Fail the promotion gate and keep the current champion active.
A candidate fails its promotion smoke test	Restore the previous champion pointer automatically.
An API pod cannot load or verify the champion	Fail readiness so the ALB sends it no traffic.
Drift or a performance regression is detected	Alert and investigate; retraining may start, but promotion remains gated.
A new service image causes failures	Roll back the EKS deployment independently of the model version.

Operational requirements

The serving API should run across multiple availability zones and scale horizontally using request volume, CPU, or latency signals. Training jobs should be idempotent so retries cannot create conflicting registry versions or promote a candidate twice.

Every prediction should record a request ID, timestamp, model version, contract version, decision, and latency for traceability. Logs and monitoring events should exclude raw identifiers and unrestricted application payloads. Registry changes remain append-only and record the actor, reason, previous champion, and new champion, with retention and access controlled through least-privilege policies.

Platform controls

Scheduled **EKS Jobs** run training against versioned S3 snapshots and write immutable artifacts back to **S3** and the registry. **CloudWatch** and **Prometheus** provide logs, metrics, and alerts, while IAM roles, secrets management, TLS, and Kubernetes RBAC restrict access to data, artifacts, and services.

Principal limitations and decisions required

Before production, owners must agree service SLOs, retention, access, audit, rollback, and disaster-recovery requirements. The local POC validates the interfaces; production still needs environment-specific security, high availability, recovery, and integration testing.